

Dia 2 estudos - IA - python

Para dicionários:

Função .items(): Retorna o item, ou seja, a chave e o valor associado

Ex: planets.get('londres', "não existe")
out: não existe

Exemplo de get: Errei!

~~planet = planets~~

~~planet massa~~

for planet in planets.dict.items

print(f"{planet}: {massa}")

Função .get(): verifica se a chave existe e retorna o segundo argumento caso não exista. Caso exista retorna o valor associado

Para Tuples:

In Podemos usar in para conferir se um item está lá dentro.

Ex: if 'mars' in planets:
print("Fato verídico")

- Só é possível modificar uma tuple se transformá-la em lista, alterar e depois transformar em tuple de novo

Ex: x = ("apple", "mango", "banana")
y = list(x)
y[1] = "Kiwi"
x = tuple(y)

+ é possível concatenar tuples com o operador "+",

Keyword del: Deleta uma tuple

Unpacking a Tuple: Podemos ^{atribuir} associar a variáveis o que tem dentro de uma tuple.

Ex:

x = ("banana", "mango", "kiwi")

x[yellow, green, hold-green] = x

print(yellow, green, hold-green)

out: banana \n mango \n kiwi \n

(*fruits, p) = x

print(yellow, fruits, p)

out: ["banana", "mango"] \n kiwi

* é possível multiplicar tuples para que copie seus itens e os adicione no final

Métodos:

.count(): retorna quantas vezes o item que vai no argumento se repete.

.index(): retorna o índice que o item do argumento se encontra (primeira aparição)

OBS: Precisa conferir se o item existe antes.

Set variable: não ordenado, imutável e indexado (não é possível acessar com keys ou valores. Não pode itens duplicados (True == 1) (False == 0).

Ex: myset = {"apple", "mango", "kiwi"}

- Assim como os outros dados integrados (list, dictionary, tuple) pode ser acessado nos laços for.

Ex: for my in myset:

- Também é possível checar a existência de itens com `in`

Métodos de adicionar em um set:

• `add()`: adiciona um item no set

Ex: `myset.add("Pineapple")`

• `update()`: adiciona qualquer objeto iterável

Métodos de remover em um set:

• `remove()`: retorna um erro se o item não existir.

• `discard()`: remove sem retornar erros, mesmo que não exista.

- Pode usar o `pop` que funciona para todos, porém é aleatória a remoção.

• `clear`: ~~em~~ esvazia o set

• `keyword del`: deleta completamente

Métodos de união com sets:

• `union()` or `update()`: Junta tudo, ^{des} ~~sen~~ de que seja objetos iteráveis

• `intersection()`: Mantém apenas as duplicatas

• `difference()`: Mantém apenas os itens do primeiro set que não estão no outro set

• `symmetric_difference()`: Mantém tudo que não é duplicado.

- Os operadores são sempre set com set

Arquivos

Abertura e fechamento de arquivos:

- Assim como em C podemos abrir arquivos usando `open()` passando como argumentos o nome do arquivo e modo de abertura. Por padrão abre em modo de leitura de texto.

Ex: `f = open("arg.txt")`

- Também podemos usar `with`. Dessa forma, o arquivo só estará aberto dentro do bloco sem precisar fechá-lo.

Ex: `with open("arg.txt", "wt") as f:
 print(f.read())`

Leitura de arquivos:

• `readline()`: Lê uma única linha.

• `read()`: Lê todo o arquivo ou a quantidade de caractere especificada no argumento.

Escrita em arquivos:

• `write()`: Passa o que está no argumento para o arquivo.

Módulo "os":

`os.remove()`: Deleta um arquivo.

`os.rmdir()`: Deleta uma pasta se ela estiver vazia.

`os.path.exists()`: Checa a existência de um arquivo.